

Lab1: 執行畫面

```
Nov 7 14:06
sudo cat /sys/kernel ~

minimal-2963 [001] ...21 1118.752880: bpf_trace_printk: BPF triggered from PID 2963 by 114064548
minimal-2963 [001] ...21 1119.753018: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1120.753983: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1121.754489: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1122.755329: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1123.755589: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1124.755944: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1125.756007: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1126.756472: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1127.757069: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1128.757736: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1129.758472: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1130.758647: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1131.759455: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1132.760008: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
minimal-2963 [001] ...21 1133.760121: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
```

```
sudo cat /sys/kernel ~

imal-2963 [001] ...21 1118.752880: bpf_trace_printk: BPF triggered from PID 2963 by 11406454
imal-2963 [001] ...21 1119.753018: bpf_trace_printk: BPF triggered from PID 2963 by 11406454
imal-2963 [001] ...21 1120.753983: bpf_trace_printk: BPF triggered from PID 2963 by 114064548
imal-2963 [001] ...21 1121.754489: bpf_trace_printk: BPF triggered from PID 2963 by 114064548
imal-2963 [001] ...21 1122.755329: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1123.755589: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1124.755944: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1125.756007: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1126.756472: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1127.757069: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1128.757736: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1129.758472: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1130.758647: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1131.759455: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1132.760008: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
imal-2963 [001] ...21 1133.760121: bpf_trace_printk: BPF triggered from PID 2963 by 114064548.
```

失敗畫面

```
0: (00) call bpf_get_current_pid_tgid#14 ; R0_w=scalar()
; int pid = bpf_get_current_pid_tgid() >> 32;
1: (77) r0 >>= 32 ; R0_w=scalar(smin=0,smax=umax=0xf
; if (pid != my_pid)
2: (18) r1 = 0xfffffaee2c1cce000 ; R1_w=map_value(map=minimal_.bss,
4: (61) r1 = *(u32 *)(r1 +0) ; R1_w=scalar(smin=0,smax=umax=0xf
; if (pid != my_pid)
5: (5d) if r1 != r0 goto pc+5 ; R0_w=scalar(smin=0,smax=umax=0xf
R1_w=scalar(smin=0,smax=umax=0xffffffff,var_off=(0x0; 0xffffffff))
; bpf_printk("BPF triggered from PID %d by 114064548.\n", pid);
6: (18) r1 = 0xffff974ec2d3b110 ; R1_w=map_value(map=minimal_.rodat
8: (b7) r2 = 41 ; R2_w=41
9: (bf) r3 = r0 ; R0_w=scalar(id=1,smin=0,smax=umax=
ff)) R3_w=scalar(id=1,smin=0,smax=umax=0xffffffff,var_off=(0x0; 0xffffffff)
10: (85) call bpf_trace_printk#6
cannot call GPL-restricted function from non-GPL compatible program
processed 9 insns (limit 1000000) max_states_per_insn 0 total_states 0 pea
-- END PROG LOAD LOG --
libbpf: prog 'handle_tp': failed to load: -22
libbpf: failed to load object 'minimal_bpf'
libbpf: failed to load BPF skeleton 'minimal_bpf': -22
Failed to load and verify BPF skeleton
➔ minimal git:(main) ×
17
18 // TODO: 把STUDENT_ID換成你的學號
19 bpf_printk("BPF triggered from PID %d by 114064548.\n", pid);
20
```

- 回答問題:

1.eBPF 相較於 kernel module 的差異在哪裡? (10 points)

eBPF 可在不修改 kernel 模組的情況下，動態插入並執行安全受控的程式邏輯，透過 verifier 驗證確保不會破壞系統；而 kernel module 直接以核心權限執行，功能強但風險高，一旦出錯可能造成 kernel panic。

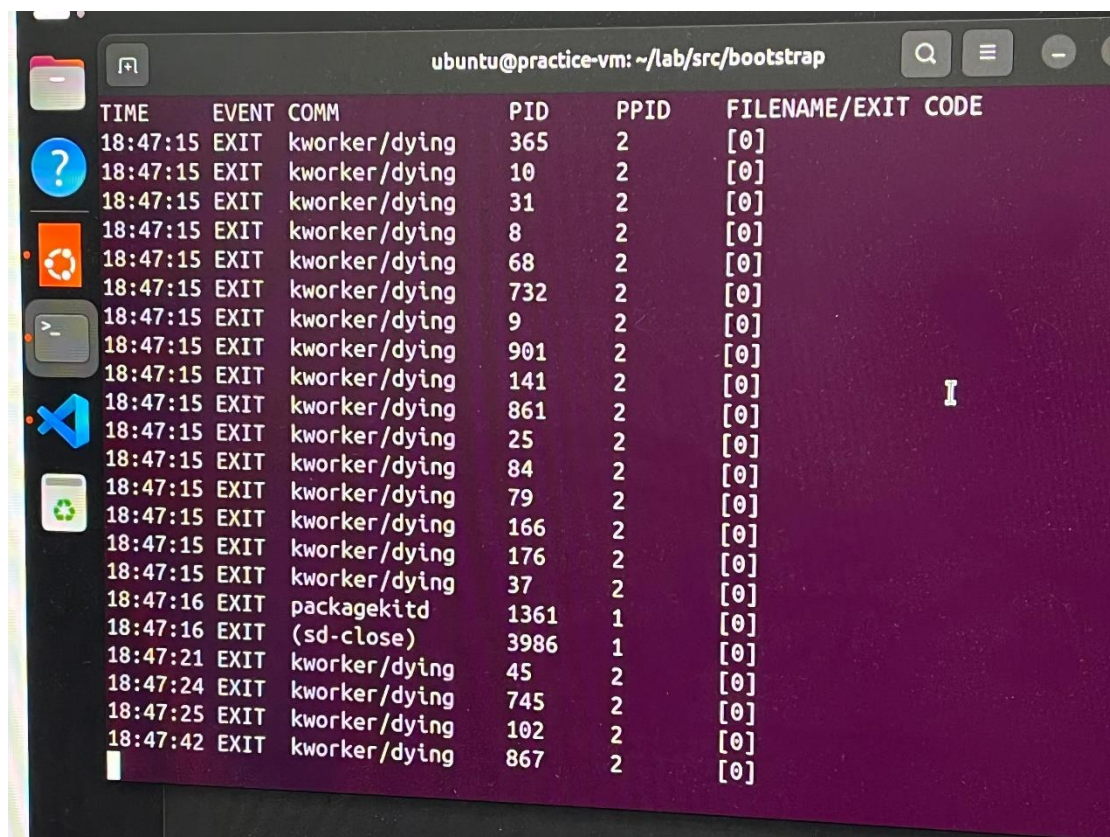
2.參考[這裡](#)，並回答為什麼註解掉 license 聲明就會導致不通過 verifier 的驗證? (5 points)

因為沒宣告 license 時，kernel 預設程式為非 GPL，用到任何 GPL 專屬 helper 時 verifier 會直接拒絕載入，判定非 GPL 程式呼叫 GPL-only 函式，因此驗證不通過

3.參考[這裡](#)，並回答 linux kernel 使用 verifier 檢查了 eBPF 程式的哪些面向來確保 eBPF 程式的安全性? (5 points)

verifier 會檢查 eBPF 程式的記憶體存取是否合法、是否有無限迴圈、函式呼叫是否合法、指令流程是否可達、以及執行堆疊與寄存器使用是否安全，以確保程式不會造成 kernel 崩潰。

Lab2:



The screenshot shows a terminal window titled 'ubuntu@practice-vm: ~/lab/src/bootstrap'. It displays a list of process exit events with columns: TIME, EVENT, COMM, PID, PPID, and FILENAME/EXIT CODE. The events are all 'EXIT' events for 'kworker/dying' processes, except for 'packagekitd' and '(sd-close)'. The exit codes are all '[0]'.

TIME	EVENT	COMM	PID	PPID	FILENAME/EXIT CODE
18:47:15	EXIT	kworker/dying	365	2	[0]
18:47:15	EXIT	kworker/dying	10	2	[0]
18:47:15	EXIT	kworker/dying	31	2	[0]
18:47:15	EXIT	kworker/dying	8	2	[0]
18:47:15	EXIT	kworker/dying	68	2	[0]
18:47:15	EXIT	kworker/dying	732	2	[0]
18:47:15	EXIT	kworker/dying	9	2	[0]
18:47:15	EXIT	kworker/dying	901	2	[0]
18:47:15	EXIT	kworker/dying	141	2	[0]
18:47:15	EXIT	kworker/dying	861	2	[0]
18:47:15	EXIT	kworker/dying	25	2	[0]
18:47:15	EXIT	kworker/dying	84	2	[0]
18:47:15	EXIT	kworker/dying	79	2	[0]
18:47:15	EXIT	kworker/dying	166	2	[0]
18:47:15	EXIT	kworker/dying	176	2	[0]
18:47:15	EXIT	kworker/dying	37	2	[0]
18:47:16	EXIT	packagekitd	1361	1	[0]
18:47:16	EXIT	(sd-close)	3986	1	[0]
18:47:21	EXIT	kworker/dying	45	2	[0]
18:47:24	EXIT	kworker/dying	745	2	[0]
18:47:25	EXIT	kworker/dying	102	2	[0]
18:47:42	EXIT	kworker/dying	867	2	[0]

1. eBPF 中的 ring buffer 跟 hash map 的差異在哪裡? (10 points)

Ring buffer 是單向、鎖 less 的事件管道，用來把 eBPF 產生的順序化事件推到 user space；Hash map 是 kernel 內的 key-value 狀態表，用於持久儲存、查找與更新 eBPF 執行時需要的資料。

2. 追蹤 bootstrap 的程式碼，並回答它是如何在程式中使用 ring buffer 跟 hash map 的? (10 points)

在 bootstrap 範例裡，eBPF 在 sched_process_exec/exit 等 hook 產生 struct event，用 ring buffer 提交到使用者程式；同時用 hash map 做 per-PID 狀態/過濾（例如記錄與查詢要追蹤的 PID 或該 PID 的臨時資料），讓 exit/exec 兩端的程式邏輯能快速查到對應的流程資訊。